



INTERNET MANAGEMENT (SNMP)



Reti di Calcolatori - Modulo 2

Introduction

- ▶ Autonomous systems include hundreds or thousands of hardware and software components interacting among them
- ▶ The concept of network management includes
 - ▶ Development, integration, and coordination of hardware, software and personnel (human)
 - ▶ Monitoring, querying, configuration and evaluation of an IP network and its resources
- ▶ The goal is to permit correct working of the network at limited costs

Introduction

- ▶ Need of software that allows to
 - ▶ Debug problems
 - ▶ Control routing
 - ▶ Find computers that violate protocol standards
- ▶ TCP/IP Internet management software and protocol

Functional Areas of Network Management

- ▶ Resource management
- ▶ Security management
- ▶ Performance management
- ▶ Configuration management
- ▶ Fault management

Management Protocols

- ▶ Early network systems used two approaches
 - ▶ Separate, parallel management network
 - ▶ Link-level management commands (WAN)
- ▶ Packet switch misbehaves
 - ▶ Network manager sends a control packet via a neighboring packet switch
 - ▶ Switch suspends normal operation
 - ▶ Network manager tests the switch

Management Protocols

- ▶ Link-level solutions work also when higher level protocols fail → Solo in reti local
- ▶ PROBLEMS
 - ▶ Multiple link level protocols
 - ▶ Several physical networks
 - ▶ Manager needs to control machine not attached to the same network
 - ▶ Internet management different from network management

TCP/IP Management Protocols

- ▶ TCP/IP pioneered running management protocols at the application layer
- ▶ Motivation: provide internet-wide capability instead of single network capability
- ▶ A manager connects to machines residing on different physical networks

TCP/IP Management Protocols

- ▶ A single manager controls heterogeneous devices
- ▶ Controlled entities may not share a common link level protocol
- ▶ A manager controls machines in arbitrary points of the Internet
 - ▶ To this aim Internet management protocol operates above the transport level

The Point About Internet Management

- ▶ *In a TCP/IP internet, a manager needs to examine and control routers and other network devices. Because such devices attach to arbitrary networks, protocols for Internet management operate at the application level and communicate using TCP/IP transport-level protocols*

Application-level Internet Management

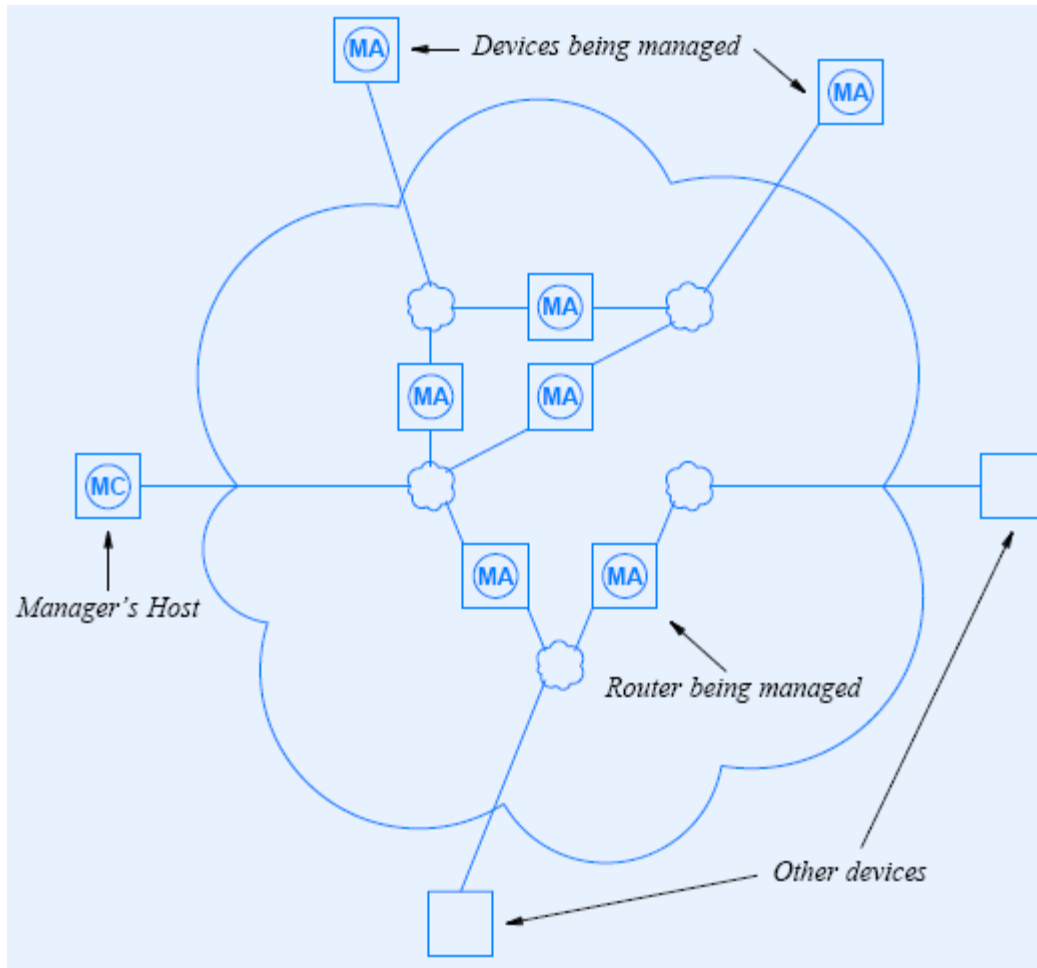
▶ Pro

- ▶ Independent from the network hardware
- ▶ Independent from the hardware on the managed machine
- ▶ Manager can control routers without direct connection
- ▶ A single set of protocols

▶ Cons

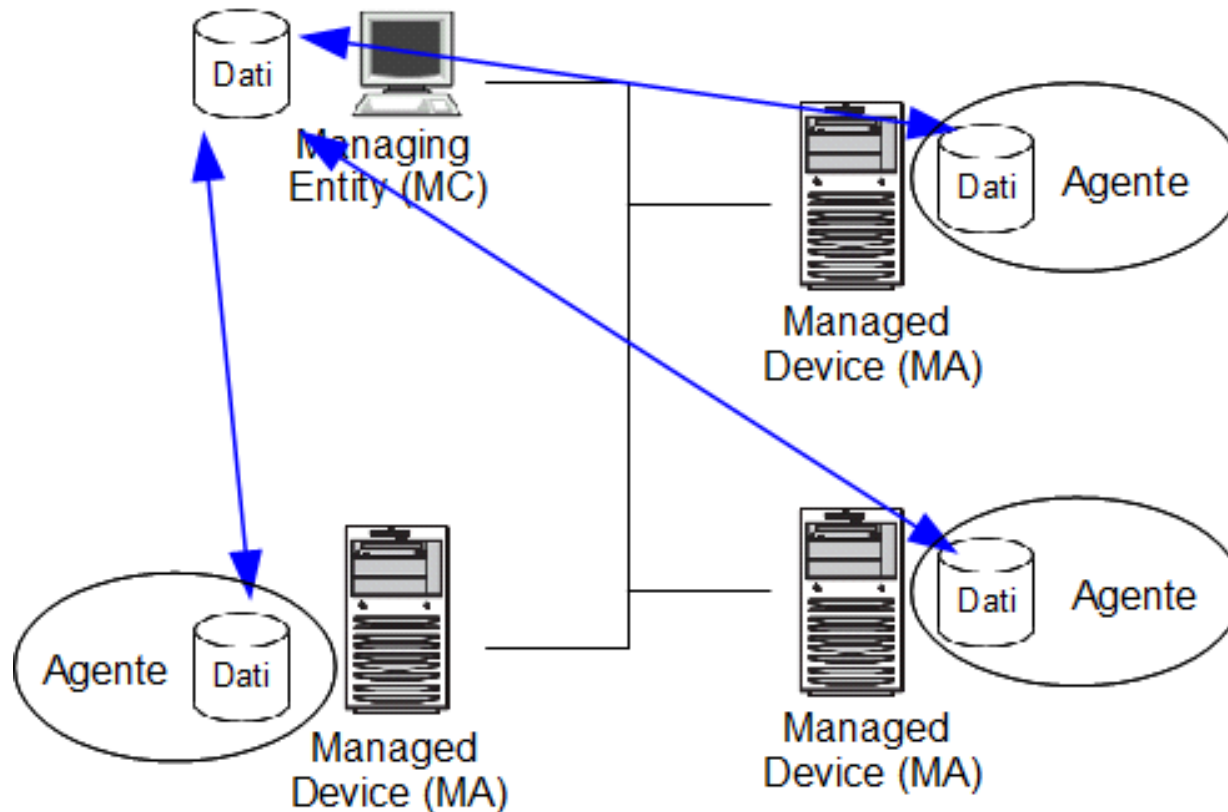
- ▶ OS, IP software, transport protocol software must work (e.g., corrupted routing table, OS crash)

Architectural Model



Architectural Model: Refined

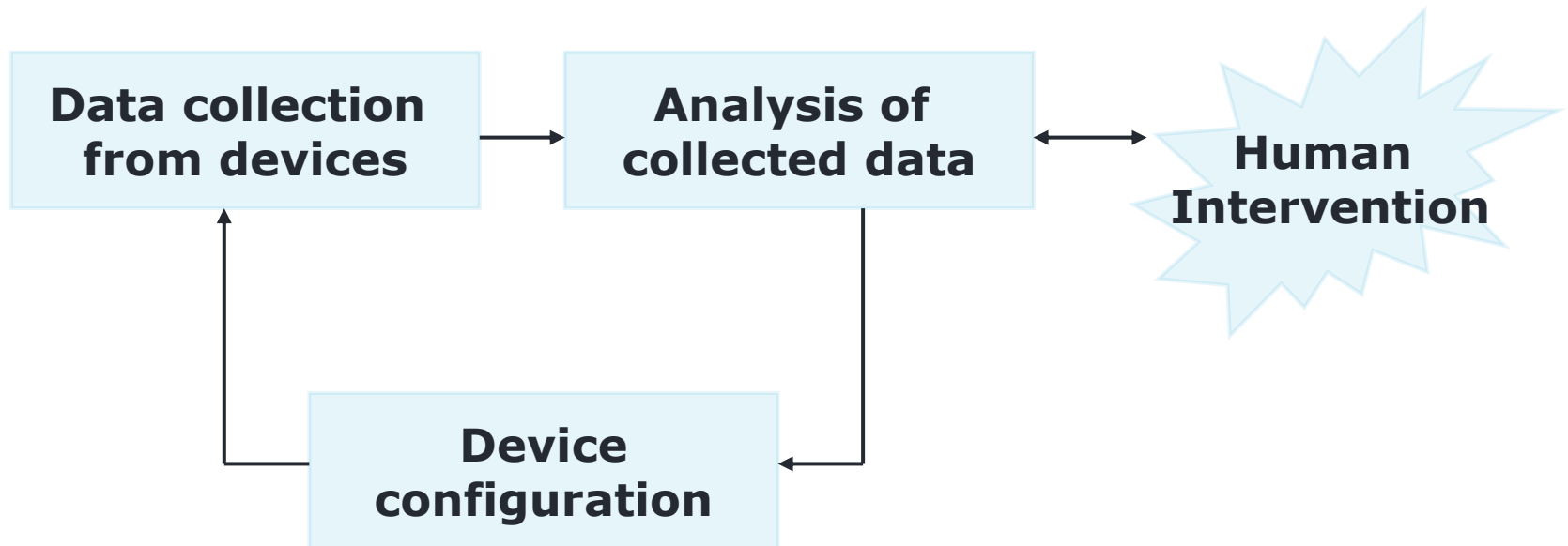
- Managed devices contain objects stored in the local databases



Terminology

- ▶ Agent (MA)
 - ▶ Runs on arbitrary systems (e.g., a router)
 - ▶ Responds to manager's requests
- ▶ Management client (MC)
 - ▶ Runs on manager's workstation
 - ▶ Controls a part of the device on the Net
 - ▶ Sends requests to agents as directed by the manager
 - ▶ Requests are query for information or commands

Management Process



Network Management

- ▶ Not all the devices in the Internet fall under the same manager
- ▶ An authentication mechanism ensures access to authorized managers only
- ▶ Multiple and fine-grained level of authorizations
 - ▶ All read, few write

TCP/IP Network Management Protocols

- ▶ Two parts, different standards
 1. *Data being managed*: specifies which data items are relevant for the device, their name and the syntax of the name
 2. *Communication of information*: defines format and meaning of messages and form of names and addresses

TCP/IP Network Management Protocols

- ▶ *Management Information Base (MIB)*
 - ▶ Which data are accessible for which devices
 - ▶ Views: agent MIB, manager MIB (union of all MIBs)
- ▶ *Structure of Management Information (SMI)*
 - ▶ Rules used to define, identify, and reference MIB variables
- ▶ *Simple Network Management Protocol (SNMP) third generation*
 - ▶ Defines message format, how to use transport protocol, the set of operations, and their meaning

MIB and SMI



MIB and SMI

- ▶ MIB (Management Information Base)
 - ▶ It represents the information storing structure (distributed) for network management data
 - ▶ It specifies the objects stored by devices, the operations allowed on them and their semantics
- ▶ SMI (Structure of Management Information)
 - ▶ Language supporting the definition of data in MIB
 - ▶ It has the goal of defining the data syntax, the specification rules, and the MIB object identification

Management Information Base (MIB)

- ▶ A device must keep control and status information that the manager can access
 - ▶ E.g., routers: statistics on packet traffic, dropped datagrams, error messages
- ▶ MIB specifies for each type of device
 - ▶ Which data items should be kept
 - ▶ Which data can be accessed
 - ▶ The operations allowed on the data and their meanings
- ▶ E.g., MIB for IP contains the number of octets arrived on the network interface

MIB Categories

MIB category	Includes Information About
system	The host or router operating system
interfaces	Individual network interfaces
at	Address translation (e.g., ARP mappings)
ip	Internet Protocol software
icmp	Internet Control Message Protocol software
tcp	Transmission Control Protocol software
udp	User Datagram Protocol software
ospf	Open Shortest Path First software
bgp	Border Gateway Protocol software
rmon	Remote network monitoring
rip-2	Routing Information Protocol software
dns	Domain Name System software

MIB Categories

- ▶ MIB for TCP/IP provides many categories for management information
- ▶ Separation between MIB definition and network management protocol gives high flexibility
 - ▶ SNMP agent software still works and is compatible with the standard in case of new MIB items
 - ▶ Same client software for different devices with different MIB items

MIB Categories

- ▶ A device without a given MIB item cannot provide information on it
- ▶ In case of a query for a not-existing MIB item
 - ▶ The query is parsed
 - ▶ An error is returned

History of MIB

- ▶ Version 1 and 2 collected variables in a single MIB
- ▶ After MIB-II, IETF decided to allow the publication of several different MIBs
- ▶ Separate RFCs to specify variables of HW bridge, ATM switch, dialup modem...

History of MIB

- ▶ Different MIBs for different types of devices each containing several variables
- ▶ Many vendors define ad-hoc MIB variables
- ▶ Most of the variables are numeric, some have complex structure
 - ▶ E.g., *ipRoutingTable* refers to a routing table

Examples of MIB Variables

MIB Variable	Category	Meaning
sysUpTime	system	Time since last reboot
ifNumber	interfaces	Number of network interfaces
ifMtu	interfaces	MTU for a particular interface
ipDefaultTTL	ip	Value IP uses in time-to-live field
ipInReceives	ip	Number of datagrams received
ipForwDatagrams	ip	Number of datagrams forwarded
ipOutNoRoutes	ip	Number of routing failures
ipReasmOKs	ip	Number of datagrams reassembled
ipFragOKs	ip	Number of datagrams fragmented
ipRoutingTable	ip	IP Routing table
icmpInEchos	icmp	Number of ICMP Echo Requests received
tcpRtoMin	tcp	Minimum retransmission time TCP allows
tcpMaxConn	tcp	Maximum TCP connections allowed
tcpInSegs	tcp	Number of segments TCP has received
udpInDatagrams	udp	Number of UDP datagrams received

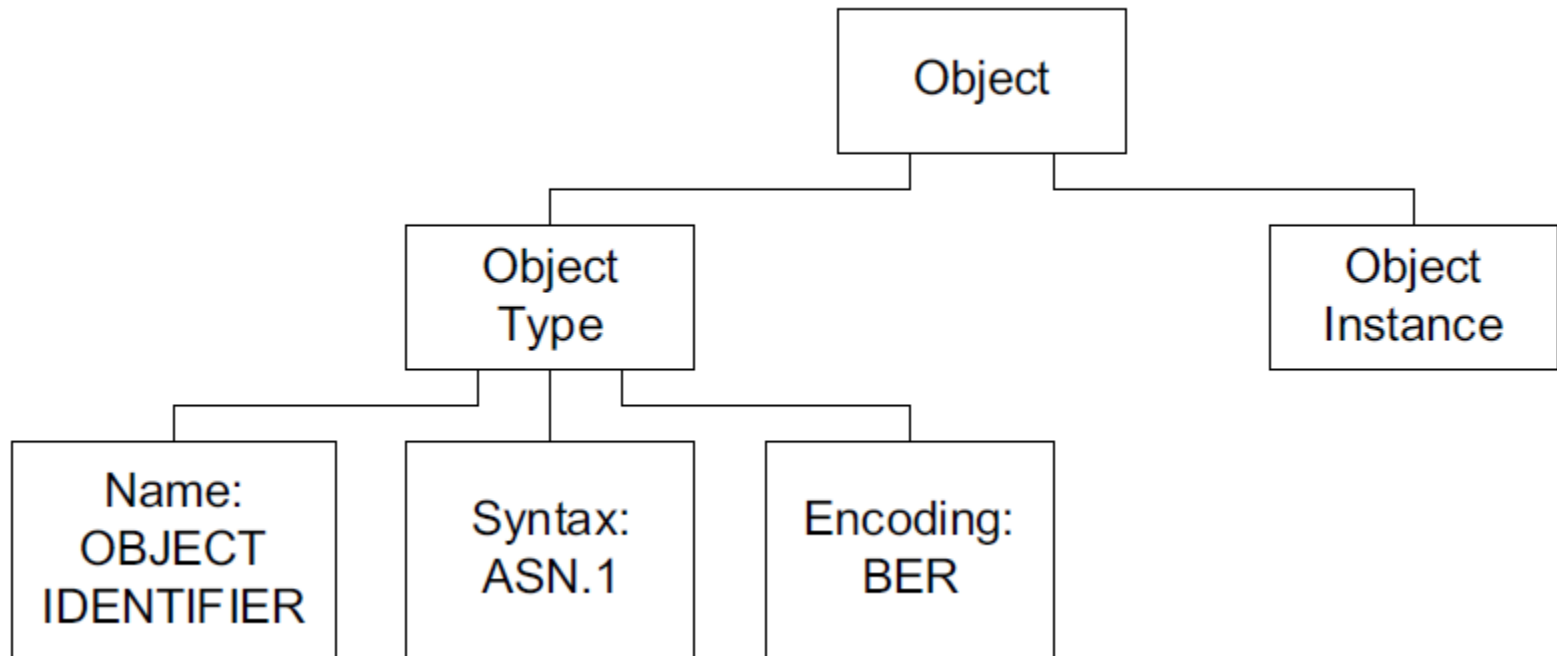
Examples of MIB Variables

- ▶ MIB variables define only logical definition of data items
- ▶ Internal data structure at device-side may differ from MIB definition
- ▶ Software agent on the device responsible to map MIB structure to device one

Structure of Management Information (SMI)

- ▶ Set of rules for defining and identifying MIB variables
 - ▶ Restrictions on the type of variables, rules on variable names, and type definition
 - ▶ How the MIB refers to tables of values
- ▶ Includes basic definitions such as
 - ▶ IP address (4-octet value)
 - ▶ Counter (integer from 0 to $2^{32} - 1$)

Managed Objects



ASN.1

- ▶ MIB variables specified and referenced using *Abstract Syntax Notation 1 (ASN.1)*
- ▶ An ISO standard and a formal language that specifies
 - ▶ A notation used in documents that human can read (user-readable format)
 - ▶ Compact encoding (format used in a message)
 - ▶ Precise and formal notation with no ambiguities (e.g., for integer values specifies form and range)

ASN.1

- ▶ Simplifies the definition of the network management protocol and guarantees interoperability
 - ▶ It defines the encoding of names and data items in the message
- ▶ Human readable format can be automatically translated in compact format

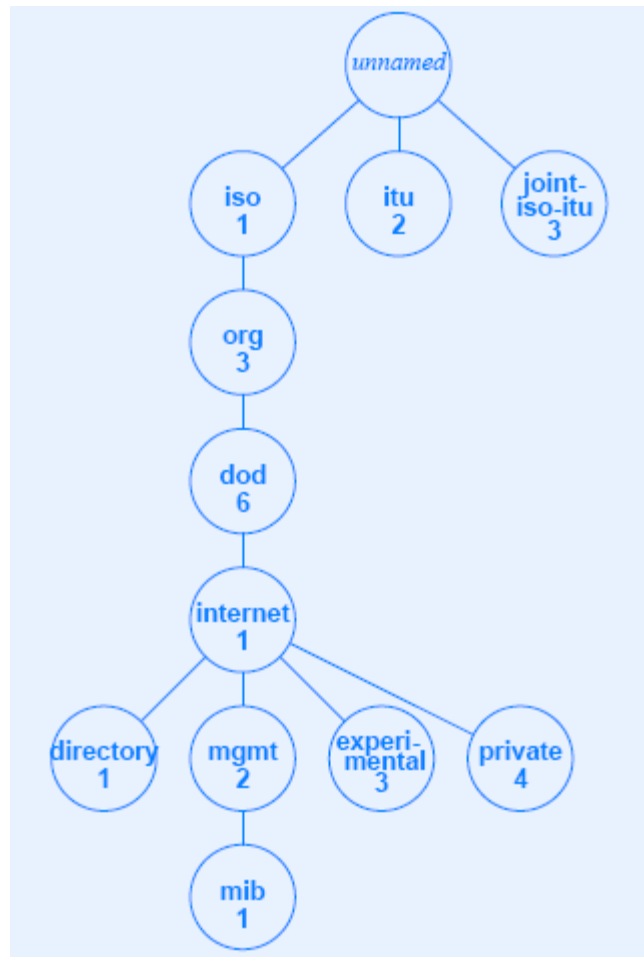
ASN.1

- ▶ *The TCP/IP network management protocols use a formal notation called ASN.1 to define names and types for variables in the management information base. The precise notation makes the form and contents of variables unambiguous*

MIB Object Names

- ▶ Name used for MIB variables taken by object identifier (OSI OID) namespace by ISO and ITU
- ▶ Namespace not restricted to network management
 - ▶ Each protocol standard document has a name
- ▶ Absolute, global, hierarchical namespace
 - ▶ Guarantee simple, unambiguous and interoperable network management protocol

Position of MIB In The Hierarchy

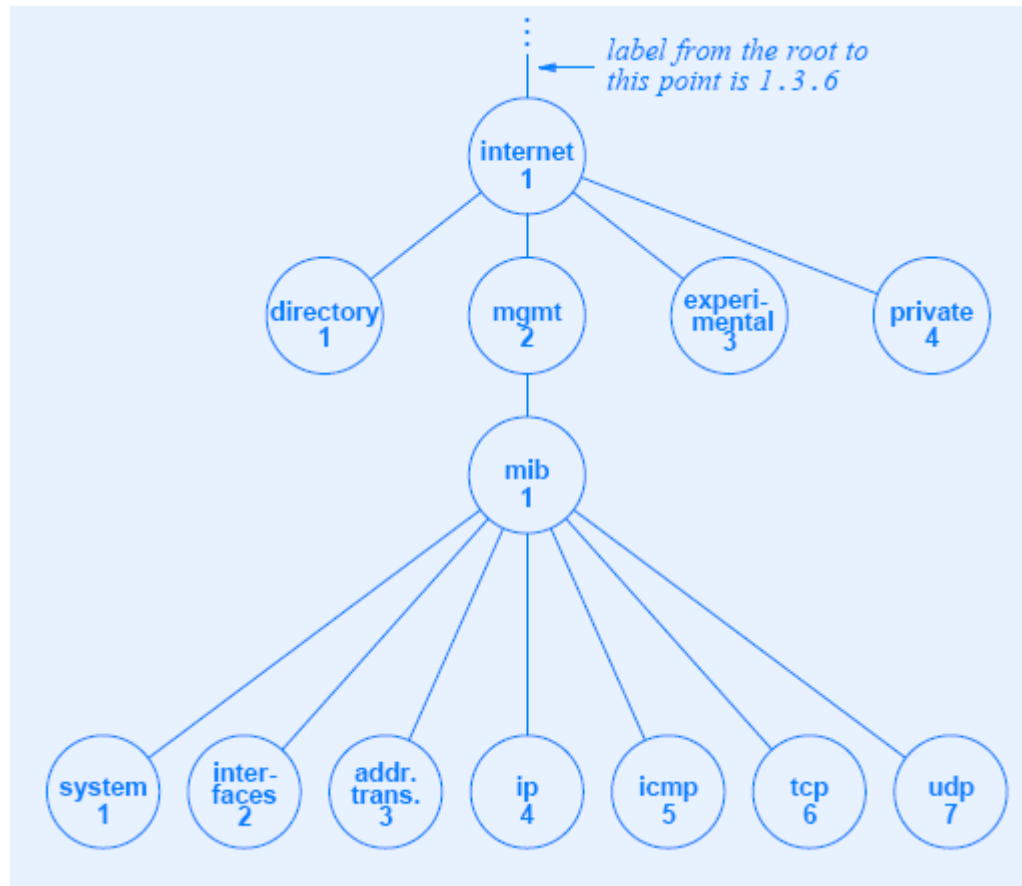


Syntactic Form

- ▶ Variable name written as sequence of labels with dot (period as delimiter)
- ▶ Numeric encoding used in messages
- ▶ Example: prefix for mib inside mgmt node

1 . 3 . 6 . 1 . 2 . 1

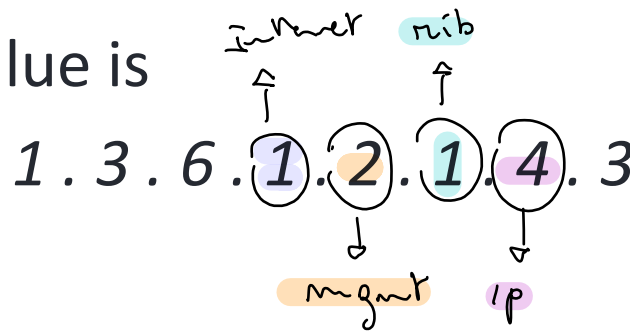
Hierarchy For TCP/IP



Naming syntax

- ▶ MIB categories are subtree of the MIB node
- ▶ Prefix for variable *ipInReceives* is
iso . org . dod . internet . mgmt . mib . ip . ipInReceives

- ▶ Numeric value is



Data Types

INTEGER

Subtype INTEGER (n1..nN)

Special case: Enumerated

INTEGER type

OCTET STRING

8-bit bytes binary and textual data

Subtypes can be specified by either range or fixed

OBJECT IDENTIFIER

Object Position in MIB

IPAddress

IP address

Counter

2-byte counter

Gauge

Not-negative number (4 byte)

TimeTick

Counter (100th of seconds)

SEQUENCE and **SEQUENCE OF**

Structure and constructor of lists of simple types



MIB Tables

- ▶ Correspond to data structures programmers think of as arrays or structs
 - ▶ *ipAddrTable* contains a list of IP addresses for each network interface
 - ▶ Such an array usually not in memory
 - ▶ Table stored in different variables connected by pointers
- ▶ MIB provides a name for the array and relies on network management software to map tables in internal variables
- ▶ ASN.1 definition uses keyword *SEQUENCE*
- ▶ *ipAddrTable* substituted by *ipAddressTable*
 - ▶ Here we refer to *ipAddrTable* for simplicity
 - ▶ <http://www.net-snmp.org/docs/mibs/ip.html>

MIB Tables: ipAddressTable

ipAddressTable OBJECT-TYPE

SYNTAX SEQUENCE OF IpAddressEntry

MAX-ACCESS not-accessible

STATUS current

DESCRIPTION

"This table contains addressing information relevant to the entity's interfaces.

This table does not contain multicast address information. Tables for such information should be contained in multicast specific MIBs, such as RFC 3019.

While this table is writable, the user will note that several objects, such as ipAddressOrigin, are not. The intention in allowing a user to write to this table is to allow them to add or remove any entry that isn't permanent. The user should be allowed to modify objects and entries when that would not cause inconsistencies within the table. Allowing write access to objects, such as ipAddressOrigin, could allow a user to insert an entry and then label it incorrectly.

Note well: When including IPv6 link-local addresses in this table, the entry must use an InetAddressType of 'ipv6z' in order to differentiate between the possible interfaces."

::= { ip 34 }

ipAddressEntry OBJECT-TYPE

SYNTAX IpAddressEntry

MAX-ACCESS not-accessible

STATUS current

DESCRIPTION

"An address mapping for a particular interface."

INDEX { ipAddressAddrType, ipAddressAddr }

::= { ipAddressTable 1 }

IpAddressEntry ::= SEQUENCE {

ipAddressAddrType	InetAddressType,
ipAddressAddr	InetAddress,
ipAddressIfIndex	InterfaceIndex,
ipAddressType	INTEGER,
ipAddressPrefix	RowPointer,
ipAddressOrigin	IpAddressOriginTC,
ipAddressStatus	IpAddressStatusTC,
ipAddressCreated	TimeStamp,
ipAddressLastChanged	TimeStamp,
ipAddressRowStatus	RowStatus,
ipAddressStorageType	StorageType

}

<https://datatracker.ietf.org/doc/rfc4293/>



MIB Tables: ipAddrTable

ipAddrTable OBJECT-TYPE

SYNTAX SEQUENCE OF IpAddrEntry

MAX-ACCESS not-accessible

STATUS deprecated

DESCRIPTION

"The table of addressing information relevant to this entity's IPv4 addresses.

This table has been deprecated, as a new IP version-neutral table has been added. It is loosely replaced by the ipAddressTable although several objects that weren't deemed useful weren't carried forward while another (ipAdEntReasmMaxSize) was moved to the ipv4InterfaceTable."

::= { ip 20 }

ipAddrEntry OBJECT-TYPE

SYNTAX IpAddrEntry

MAX-ACCESS not-accessible

STATUS deprecated

DESCRIPTION

"The addressing information for one of this entity's IPv4 addresses."

INDEX { ipAdEntAddr }

::= { ipAddrTable 1 }

IpAddrEntry ::= SEQUENCE {

ipAdEntAddr IpAddress,

ipAdEntIfIndex INTEGER,

ipAdEntNetMask IpAddress,

ipAdEntBcastAddr INTEGER,

ipAdEntReasmMaxSize INTEGER

}

<https://datatracker.ietf.org/doc/rfc4293/>



MIB Tables: SEQUENCE

- ▶ Structured data type: sequence

ipAddrTable ::= SEQUENCE OF IpAddrEntry *→ array d' struct*

IpAddrEntry ::= SEQUENCE {

ipAdEntAddr	IpAddress,
ipAdEntIfIndex	INTEGER,
ipAdEntNetMask	IpAddress,
ipAdEntBcastAddr	IpAddress,
ipAdEntReasmMaxSize	INTEGER (0..65535)

}

	Object	OBJECT IDENTIFIER	ObjectSyntax
1	ipAdEntAddr	{ipAddrEntry 1}	IpAddress
2	ipAdEntIfIndex	{ipAddrEntry 2}	INTEGER
3	ipAdEntNetMask	{ipAddrEntry 3}	IpAddress
4	ipAdEntBcastAddr	{ipAddrEntry 4}	INTEGER
5	ipAdEntReasmMaxSize	{ipAddrEntry 5}	INTEGER
6	ipAddrEntry	{ipAddrTable 1}	SEQUENCE

MIB Tables: SEQUENCE OF

- ▶ SEQUENCE OF <entry>
 - ▶ where <entry> is a list constructor

ipAddrTable ::= SEQUENCE OF IpAddrEntry

	Object Name	OBJECT IDENTIFIER	Syntax
7	ipAddrTable	{ip 20}	SEQUENCE OF

MIB Tables: Instance

ipAddrEntry OBJECT-TYPE

SYNTAX IpAddrEntry

MAX-ACCESS not-accessible

STATUS current

DESCRIPTION

"The addressing information for one of this entity's IP addresses."

INDEX { ipAdEntAddr }

::= { ipAddrTable 1 }

- ▶ ipAddrEntry is in ipAddrTable and has numeric value 1
 - ▶ ipAddrEntry { ipAddrTable 1 }
- ▶ ipAdEntNetMask is in ipAddrEntry and has numeric value 3
 - ▶ ipAdEntNetMask {ipAddrEntry 3 }
- ▶ Substantial difference in array management between programmers and network management software

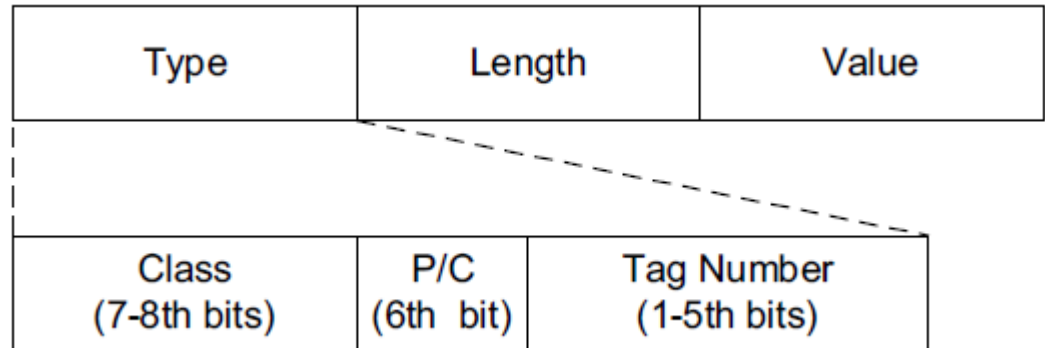
Encoding

- ▶ SMI uses: Basic Encoding Rules (BER) to encode data to be transmitted over the network
- ▶ The relationship between ASN.1 and BER parallels that of source code and machine code
- ▶ All SNMP messages are converted / serialized from ASN.1 notation into smaller, binary data (BER)

Encoding

- ▶ Basic Encoding Rules (BER)
 - ▶ Type, Length, and Value (TLV)

- ▶ Class
 - ▶ 00 UNIVERSAL
 - ▶ 01 APPLICATION
 - ▶ 10 Context-specific
 - ▶ 11 Private



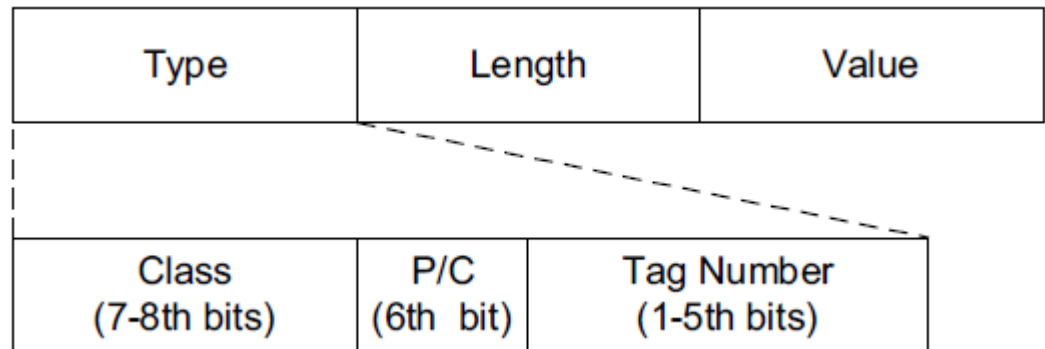
- ▶ P/C – Primitive (simple) or Composite (structured)
- ▶ Examples:
 - ▶ BOOLEAN – UNIVERSAL 1 -- 1 is a Tag
 - ▶ INTEGER – UNIVERSAL 2 -- 2 is a Tag
- ▶ The number subfield further divides simple or structured data into sub-groups

Encoding

- ▶ Basic Encoding Rules (BER)
- ▶ Type, Length, and Value (TLV)

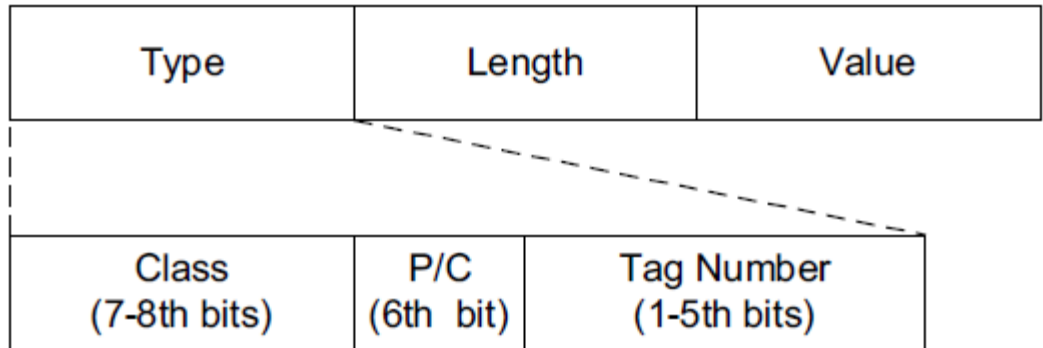
- ▶ SNMP Data Types and Tags

Type	Tag
OBJECT IDENTIFIER	UNIVERSAL 6
SEQUENCE	UNIVERSAL 16
IpAddress	APPLICATION 0
Counter	APPLICATION 1
Gauge	APPLICATION 2
TimeTicks	APPLICATION 3
Opaque	APPLICATION 4



Encoding

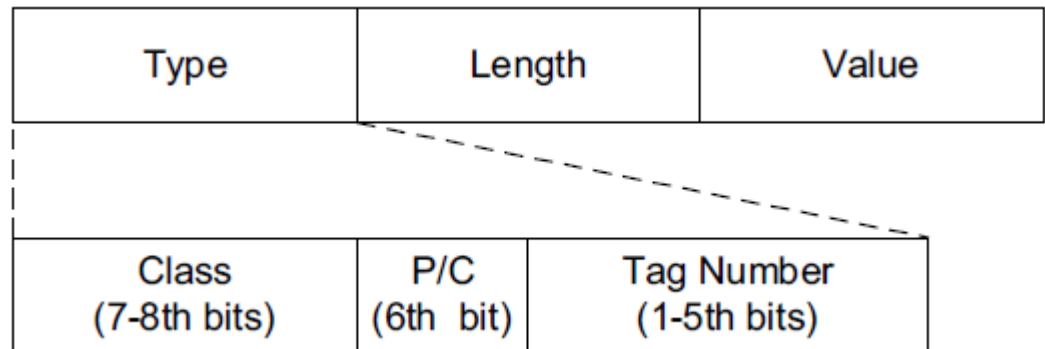
- ▶ Basic Encoding Rules (BER)
- ▶ Type, Length, and Value (TLV)



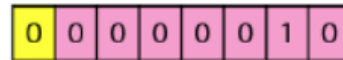
- ▶ Length
 - ▶ Length is 1 or more bytes
 - ▶ If the length can be defined with 1 byte, the most significant bit must be 0. The other 7 bits define the length of the data.
 - ▶ If the length must be defined with more than 1 byte, the most significant bit must be 1.
 - ▶ The other 7 bits define the number of bytes needed to define the length of the data

Encoding

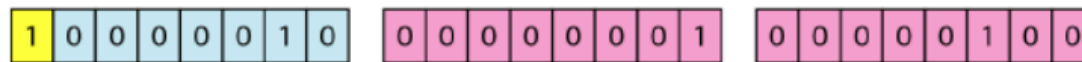
- ▶ Basic Encoding Rules (BER)
- ▶ Type, Length, and Value (TLV)



- ▶ Length (example)



a. The colored part defines the length (2).



b. The shaded part defines the length of the length (2 bytes);
the colored bytes define the length (260 bytes).

Encoding: Example

► Data Types

Data Type	Class	Format	Number	Tag (Binary)	Tah (Hex)
INTEGER	00	0	00010	00000010	02
OCTET STRING	00	0	00100	00000100	04
OBJECT IDENTIFIER	00	0	00110	00000110	06
NULL	00	0	00101	00000101	05
Sequence, Sequence of	00	1	10000	00110000	30
IPAddress	01	0	00000	01000000	40
Counter	01	0	00001	01000001	41
Gauge	01	0	00010	01000010	42
TimeTicks	01	0	00011	01000011	43
Opaque	01	0	00100	01000100	44
...	...	...	...	...	...



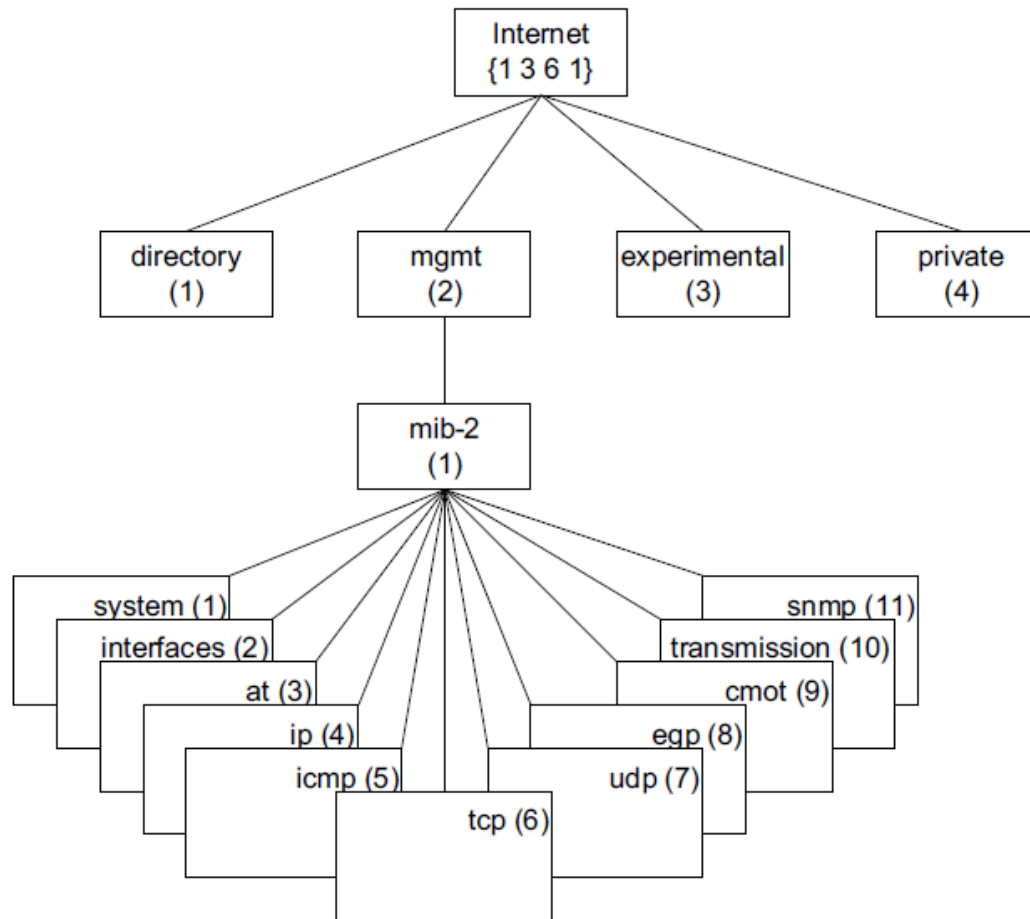
Encoding: Example

► Encoding 02 01 03
 INTEGER len=1 vers=3

- INTEGER: 00000010 -> 2
- Length: 00000001 -> 1
- Value: 00000011 -> 3

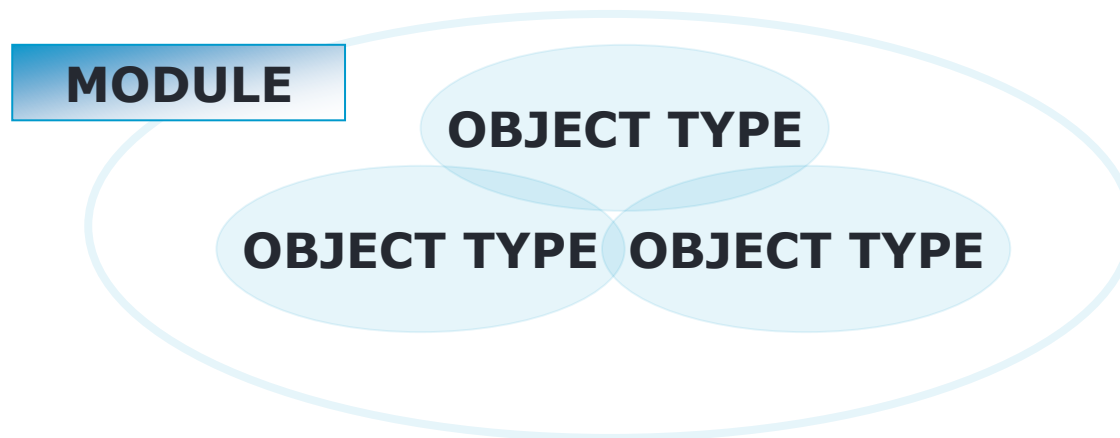
Data Type	Class	Format	Number	Tag (Binary)	Tag (Hex)
INTEGER	00	0	00010	00000010	02
OCTET STRING	00	0	00100	00000100	04
OBJECT IDENTIFIER	00	0	00110	00000110	06
NULL	00	0	00101	00000101	05
Sequence, Sequence of	00	1	10000	00110000	30
IPAddress	01	0	00000	01000000	40
Counter	01	0	00001	01000001	41
Gauge	01	0	00010	01000010	42
TimeTicks	01	0	00011	01000011	43
Opaque	01	0	00100	01000100	44
...	...	...	...	...	...

MIB Object Identifier



MIB Definition: Modules and Objects

- ▶ They are specified using SMI through module-identity (100 standard MIBs, additional vendor-defined MIBs)
- ▶ Object-type specify the structure, status and semantics of a managed-object (MIB variable)
- ▶ MODULE-IDENTITY: group related objects in a module



MODULE-IDENTITY: Example

ipMIB MODULE-IDENTITY

LAST-UPDATED "941101000Z"

ORGANIZATION "IETF SNpv2 Working Group"

CONTACT-INFO "Keith McCloghrie..."

DESCRIPTION

"Il modulo MIB per gestire implementazioni IP e ICMP
escludendo però la gestione delle rotte IP."

REVISION "019331000Z"

.....

::={mib-2 48}

UDP MODULE-ENTITY

OID	Name	Type	Description
1.3.6.1.2.1.7.1	UDPInDatagr	Counter32	Number of datagrams delivered to this entity
1.3.6.1.2.1.7.2	UDPNoPorte	Counter32	Number of datagrams that cannot be delivered because no applications is listening on the destination port
1.3.6.1.2.1.7.3	UDPInError	Counter32	Number of datagrams that cannot be delivered for other reasons
1.3.6.1.2.1.7.4	UDPDatagrInv	Counter32	Number of sent datagrams
1.3.6.1.2.1.7.5	UDPTable	Sequence	An element for each port used by the application, it returns the port number and IP address



OBJECT-TYPE: Example

udpInDatagrams OBJECT-TYPE

SYNTAX Counter32

MAX-ACCESS read-only

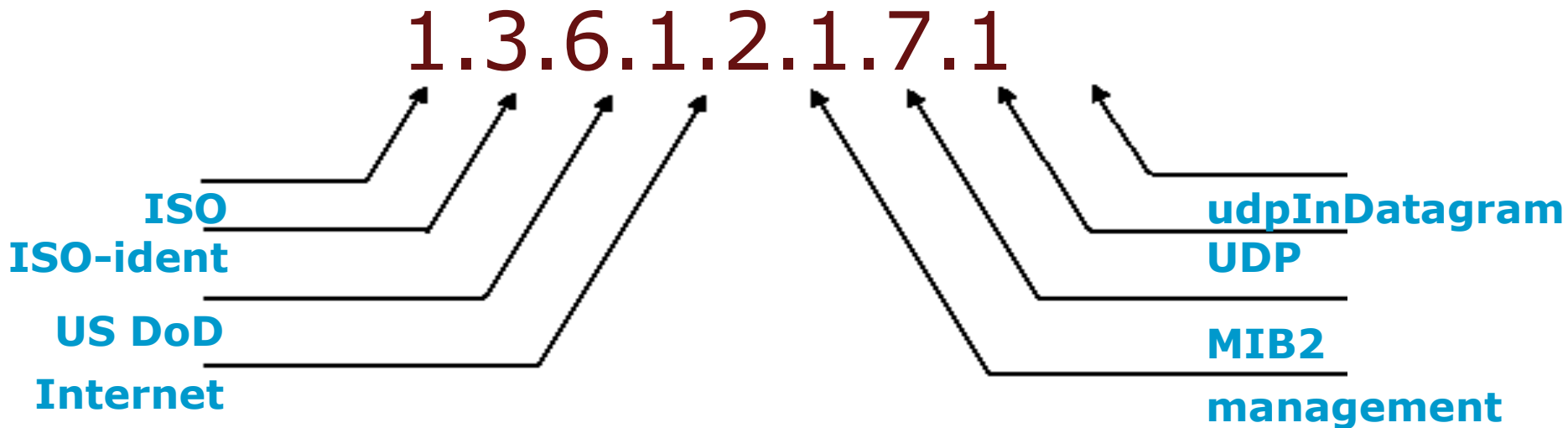
STATUS current

DESCRIPTION

"The total number of UDP datagrams delivered to UDP users."

::= { udp 1 }

Example of MIB Variable



Instance of MIB Variables

- ▶ MIB variables have a suffix when used in the network management protocol
 - ▶ Simple variable as suffix 0
 - ▶ oid of the instance is <id-leaf>.0
- ▶ Instance of variable *udpInDatagrams*
1.3.6.1.2.1.7.1.0
- ▶ No way to guess the numeric value or suffix
 - ▶ Consult the published standard or
 - ▶ Keep a table of equivalence

Instance of Tabular Objects

- ▶ Programmers
 - ▶ Set of elements that have an index: name[x]
- ▶ ASN.1
 - ▶ No indices but a suffix to select a specific element
 - ▶ E.g., the suffix for IP address table is the IP address and is appended to MIB variable name

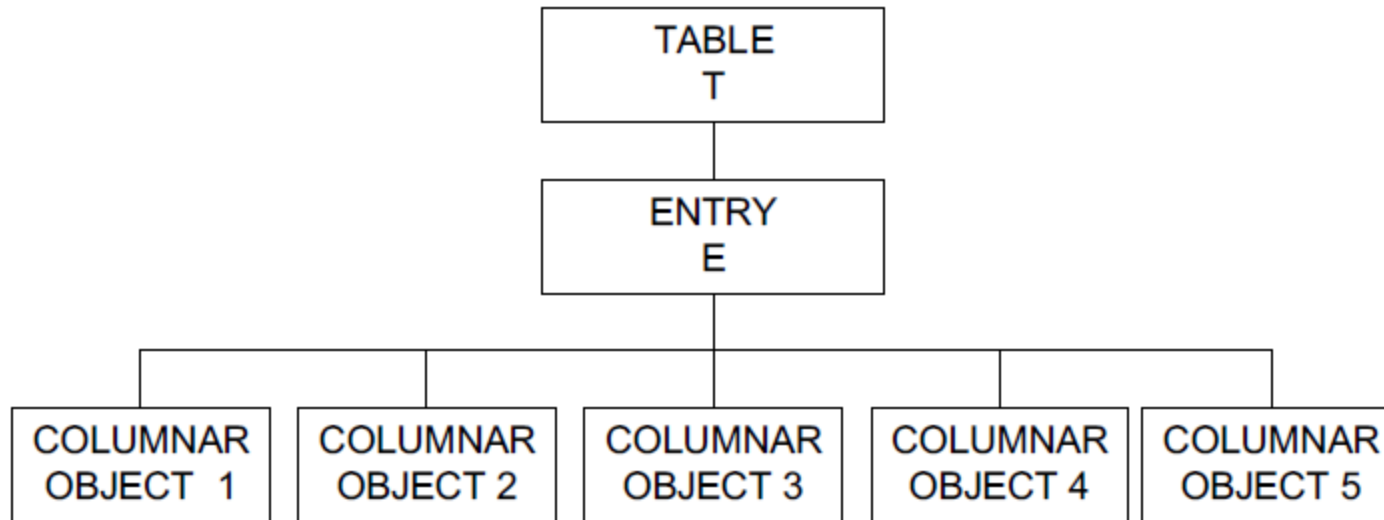
Instance of Tabular Objects

- ▶ Tabular Objects (e.g. ipAddrTable)
 - ▶ Type is a SEQUENCE OF xx and xx an OBJECT-TYPE with a field INDEX
 - ▶ Different instances determined by the value of the index,
 - ▶ OID of the instance is $\text{OID}(xx).\text{index}$
 - ▶ In case of n indexes is $\text{OID}(xx).\text{index1} \dots \text{indexn}$
 - ▶ Instances grouped in a table
- ▶ Tabular objects
 - ▶ A SEQUENCE OF rows (i.e., C array)
- ▶ Rows composed of various elements
 - ▶ Row is a SEQUENCE of elements (i.e., C record)
 - ▶ One or several elements of the SEQUENCE has an INDEX field

Instance of Tabular Objects

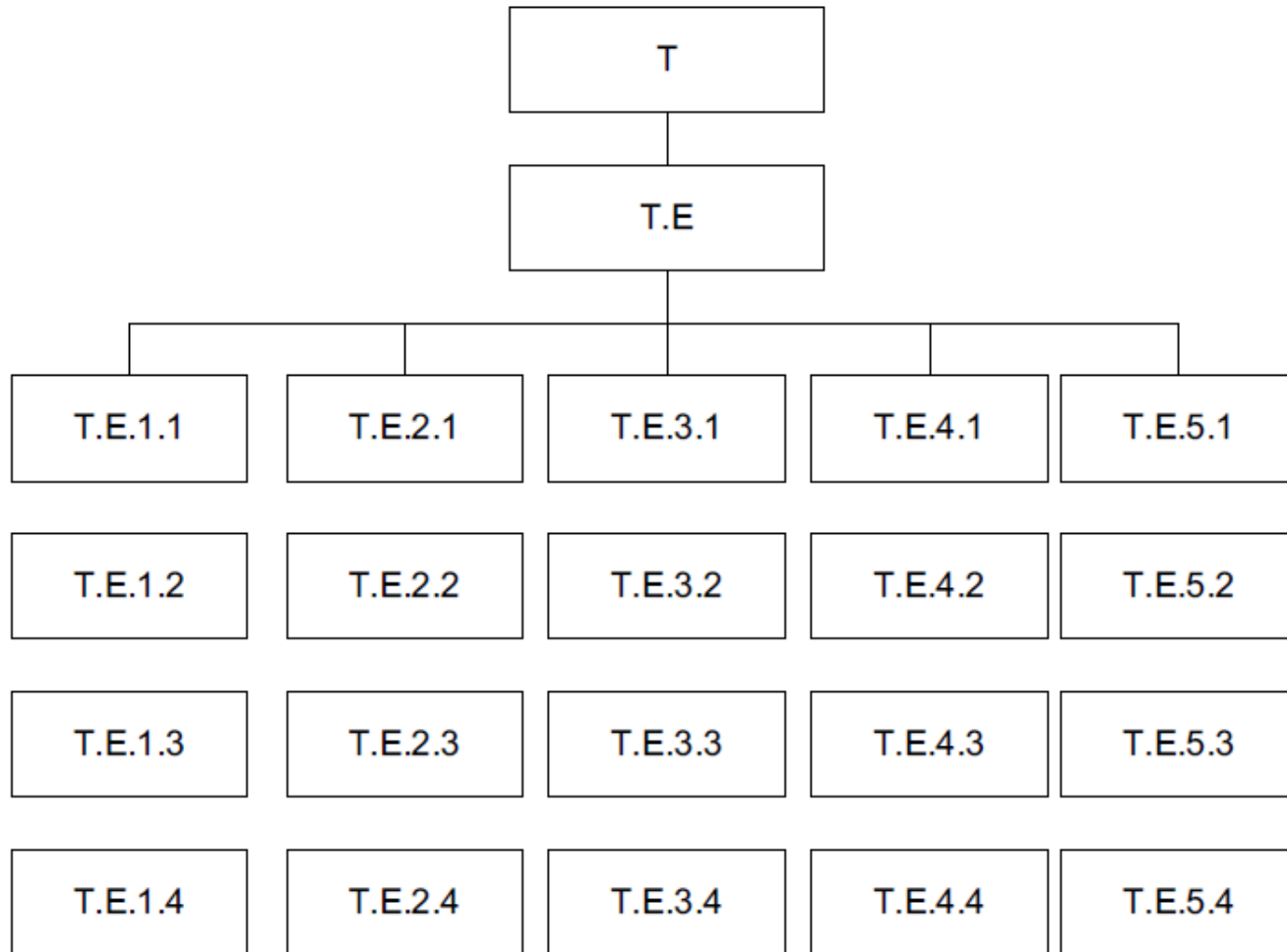
- ▶ A group of objects
- ▶ Also called tabular objects
- ▶ Can be represented by a table with
 - ▶ Columns of objects
 - ▶ Rows of instances
- ▶ Table of Objects -> List of Objects -> Objects

Tabular Representation



- ▶ TABLE T and ENTRY E are logical objects
 - ▶ They define the grouping and are not accessible
- ▶ Columnar objects are accessible objects that represent the attributes
- ▶ Each instance of E is a row of columnar objects 1 through 5
- ▶ Multiple instances of E are represented by multiple rows

Tabular Representation



Search in Tables: Recall ipAddrEntry

ipAddrEntry OBJECT-TYPE

SYNTAX IpAddrEntry

MAX-ACCESS not-accessible

STATUS current

DESCRIPTION

"The addressing information for one of this entity's IP addresses."

INDEX { ipAdEntAddr }

::= { ipAddrTable 1 }

Search in Tables

- ▶ Example: select the network mask of entry corresponding to IP address 128.10.2.3
 - ▶ *iso . org . dod . internet . mgmt . mib . ip . ipAddrTable . ipAddrEntry . ipAdEntNetMask . 128.10.2.3*
 - ▶ *1 . 3 . 6 . 1 . 2 . 1 . 4 . 20 . 1 . 3 . 128.10.2.3*
- ▶ This solution allows to search in a table without knowing the number of items

Search in Tables

ipNetToMediaTable OBJECT-TYPE

SYNTAX SEQUENCE OF IpNetToMediaEntry

ACCESS not-accessible

STATUS mandatory

DESCRIPTION

"The IP Address Translation table used for mapping
from IP addresses to physical addresses."

::= { ip 22 }

IpNetToMediaEntry ::=

SEQUENCE {

ipNetToMediaIfIndex INTEGER,

ipNetToMediaPhysAddress PhysAddress,

ipNetToMediaNetAddress IpAddress,

ipNetToMediaType INTEGER

}

ipNetToMediaEntry OBJECT-TYPE

SYNTAX IpNetToMediaEntry

ACCESS not-accessible

STATUS mandatory

DESCRIPTION

"Each entry contains one IpAddress to 'physical'
address equivalence."

**INDEX { ipNetToMediaIfIndex,
ipNetToMediaNetAddress }**

::= { ipNetToMediaTable 1 }



Search in Tables

- ▶ Suppose that the agent's IP net-to-media table has three entries:

Interface-Number	Physical-Address	Network-Address	Type
1	00:00:10:01:23:45	10.0.0.51	static
1	00:00:10:54:32:10	9.2.3.4	dynamic
2	00:00:10:98:76:54	10.0.0.15	dynamic

- ▶ With multiple indexes, each index is attached as suffix in order
 - ▶ <https://datatracker.ietf.org/doc/html/rfc1448#section-4.2.2.1>
 - ▶ ipNetToMediaPhysAddress.1.9.2.3.4,
 - ▶ ipNetToMediaType.1.9.2.3.4

MIB and Implementation

- ▶ Although they appear to specify details about data structures, MIB standards do not dictate the implementation
- ▶ MIB is a uniform, virtual interface to access data
- ▶ An agent translates between MIB and internal representation

Examples of MIB Modules

▶ IP-MIB

- ▶ ipAddressTable
- ▶ <https://datatracker.ietf.org/doc/html/rfc4293>
- ▶ <http://www.net-snmp.org/docs/mibs/ip.html>

SNMP



Common Management Information Protocol (CMIP)

- ▶ CMIP is a generic protocol for management information retrieval
- ▶ Used to monitor and check a network compliant to ISO specifications
- ▶ Defined in the 80s to be the management standard
- ▶ Limited success: development was too slow

SNMP

- ▶ SNMP (Simple Network Management Protocol) defined in 1988
 - ▶ Extend SGMP (Simple Gateway Management Protocol)
- ▶ Rapid development, it becomes increasingly complex and with many functionalities
- ▶ It has been revised and is now at version 3
 - ▶ Advanced security and management functionalities

Simple Network Management Protocol (SNMP)

- ▶ Specifies communication between manager's workstation and managed device
 - ▶ Form and meaning of the messages
 - ▶ Representation of name and values in the messages
- ▶ Defines administrative relations among routers (e.g., authentication)
- ▶ Uses *fetch-store* paradigm
 - ↳ Come GET-SET

Simple Network Management Protocol (SNMP)

▶ SNMP and OSI Model

#	Layer	Protocol/Standard
7	Application Layer	Management and Agent APIs SNMP
6	Presentation Layer	ASN.1 and BER
5	Session Layer	RPC and NetBIOS
4	Transport Layer	TCP and UDP
3	Network Layer	IP and IPX
2	Datalink Layer	Ethernet, Token Ring, ...
1	Physical Layer	

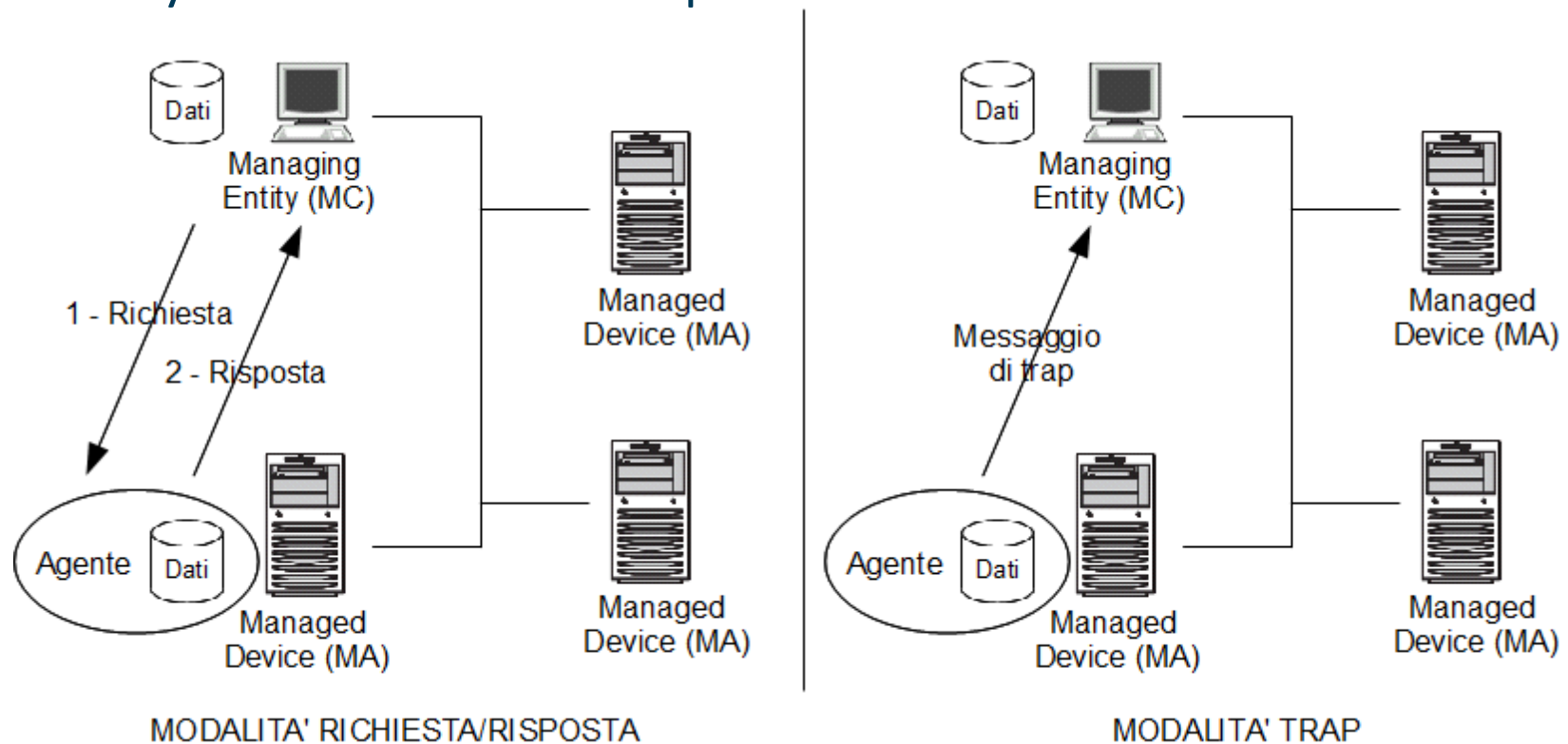
▶ UDP Port 161 - SNMP Messages

▶ UDP Port 162 - SNMP Trap Messages



Simple Network Management Protocol (SNMP)

- ▶ Two modes for data collection
 - ▶ Synchronous mode: request/response
 - ▶ Asynchronous mode: trap



Simple Network Management Protocol (SNMP)

- ▶ SNMP does not define a large set of commands
 - ▶ Reboot, add-delete routes, disable-enable network interface, remove cached address binding
 - ▶ A command for each operation increases complexity
- ▶ SNMP based on *fetch-store* paradigm
 - ▶ One command to **fetch** a value
 - ▶ One command to **store** a value
- ▶ Other operations defined as a side-effect
 - ▶ E.g., *reboot* as a variable containing `time_until_next_reboot = 0`

→ E.g. → Reboot è la modifica
del valore del time
fino al next reboot

Fetch and Store

▶ Advantages

- ▶ *Stability*: fixed definition also in case of new MIB variables and operations
 - ▶ *Simplicity*: avoid the complexity of ad-hoc commands
 - ▶ *Flexibility*: it can accomodate arbitrary commands in an elegant framework
-
- ▶ This solution is hidden from the manager point of view (e.g., reboot)
 - ▶ Graphical interface like other protocols
 - ▶ Point and click style of interaction

Operations That SNMP Supports

To iterate in a table
without suffix

Fetch

Retrieve many data
items in a single request

Command	Meaning
get-request	Fetch a value from a specific variable
get-next-request	Fetch a value without knowing its exact name
get-bulk-request	Fetch a large volume of data (e.g., a table)
response	A response to any of the above requests
set-request	Store a value in a specific variable
inform-request	Reference to third-part data (e.g., for a proxy)
snmpv2-trap	Reply triggered by an event
report	Undefined at present

Store

Server notifies
the client

- ▶ Atomic operations
- ▶ The server support all or nothing approach

Operations that SNMP Supports

- ▶ Get-request and set-request are the basic fetch and store operations
- ▶ Response provides the reply
- ▶ Trap allows the manager to instruct servers to send a message after a certain event (e.g., a failure)

Get-next-request

- ▶ ASN.1 uses a suffix to identify an individual element in the table
- ▶ What happen if the client does not know the suffix???

Get-next-request

- ▶ ASN.1 uses a suffix to identify an individual element in the table
- ▶ What happen if the client does not know the suffix???
- ▶ Get-next-request operation is used to iterate through the table

Get-next-request

- ▶ At the first request the client releases a valid prefix of a table *→ Solo prefisso della tabella*
- ▶ The server releases the first element in the table
- ▶ At the second request the client releases the first element in the table
- ▶ The server releases the second element in the table in lexicographical order
- ▶ ... *→ case ciclo for $0 \dots n$*

Get-next-request

- ▶ IpAddrTable uses the IP addresses as index
- ▶ Example: select the network mask *ipAdEntNetMask* of the first entry using the prefix
 - ▶ *iso . org . dod . internet . mgmt . mib . ip . ipAddrTable . ipAddrEntry . ipAdEntNetMask*
 - ▶ *1 . 3 . 6 . 1 . 2 . 1 . 4 . 20 . 1 . 3*
- ▶ Use the full object identifier of the first element to search the second...

SNMP Message Format

- ▶ Defined using ASN.1 notation → Sia per variabili che per messaggi
 - ▶ Difficult for human to encode
- ▶ Similar to BNF grammar
 - ▶ A language to express formal languages
- ▶ SNMP messages do not have fixed fields

Example ASN.1 Definition

SNMPv3Message ::=

SEQUENCE {

Header { msgVersion INTEGER (0..2147483647),
-- note: version number 3 is used for SNMPv3

Signature { msgGlobalData HeaderData,
msgSecurityParameters OCTET STRING,

Content
control { msgData ScopedPduData
-- payload

}



Definition Of HeaderData Area In SNMP Message

```
HeaderData ::= SEQUENCE {  
    msgID INTEGER (0..2147483647),  
        -- used to match responses with requests  
    msgMaxSize INTEGER (484..2147483647),  
        -- maximum size reply the sender can accept  
    msgFlags OCTET STRING (SIZE(1)),  
        -- Individual flag bits specify message characteristics  
        -- bit 7 authorization used  
        -- bit 6 privacy used  
        -- bit 5 reportability (i.e., a response needed)  
    msgSecurityModel INTEGER (1..2147483647)  
        -- determines exact format of security parameters that follow  
}
```

ScopedPduData

- ▶ ASN.1 uses *CHOICE* keyword for a *discriminated union*

```
ScopedPduData ::= CHOICE {  
  plaintext ScopedPDU,      — A o u n s o e' e l r n  
  encryptedPDU OCTET STRING  
    --encrypted ScopedPDU value  
}
```

ScopedPDU

- ▶ Contains the real operations, that is, PDU (protocol data unit) structure
 - ▶ Request of a client or response of a server

```
ScopedPDU ::= SEQUENCE {  
    contextEngineID OCTET STRING,  
    contextName OCTET STRING,  
    data ANY -- e.g., a PDU (next slide)  
}
```


PDU

- ▶ PDU is a choice of existing operations
 - ▶ Consists of one of the 8 types

```
PDU ::= CHOICE {  
    get-request GetRequest-PDU,  
    get-next-request GetNextRequest-PDU,  
    get-bulk-request GetBulkRequest-PDU,  
    response Response-PDU,  
    set-request SetRequest-PDU,  
    inform-request InformRequest-PDU,  
    snmpv2-trap SNMPv2-Trap-PDU,  
    report Report-PDU,  
}
```

Example

```
GetRequest-PDU ::= [0]
  IMPLICIT SEQUENCE {
    request-id
      RequestID,
    error-status
      ErrorStatus,
    error-index
      ErrorIndex,
    variable-bindings
      VarBindList
  }
```

- ▶ Error-status and error-index are 0 in case of request
- ▶ VarBindList is a sequence of pair object name-value
 - ▶ Value=null in the simplest case

Example

ASN.1 constructs used frequently:

```
RequestID ::=
    INTEGER
```

```
ErrorStatus ::=
    INTEGER {
        noError(0),
        tooBig(1),
        noSuchName(2),
        badValue(3),
        readOnly(4)
        genErr(5)
    }
```

```
ErrorIndex ::=
    INTEGER
```

```
VarBind ::=
    SEQUENCE {
        name
        ObjectName,

        value
        ObjectSyntax
    }
```

```
VarBindList ::=
    SEQUENCE OF
    VarBind
```



Example

- ▶ See attached pdf (snmp-example.pdf)

New features

- ▶ SNMPv3
 - ▶ Changes in the areas of security and administration
 - ▶ Designed to have general and flexible security policies
 - ▶ Designed to make administration of security easy (remote administration)
- ▶ Security support (with generality and flexibility)
 - ▶ Message authentication, privacy, authorization, and view-based access control
 - ▶ Security systems easy to configure and change also using remote connections

Software for network management

- ▶ Proprietary applications using standard protocols (e.g., SNMP)
- ▶ Some examples
 - ▶ CiscoWorks (Cisco)
 - ▶ OpenView (Hewlett Packard)
 - ▶ Solstice (Sun)

Summary

- ▶ Network management protocols allow to monitor and control routers and hosts
- ▶ TCP/IP management software executes as application programs and uses internet transport protocols

Summary

- ▶ Management Information Base (MIB) specifies set of variables that can be accessed
- ▶ Structure Of Management Information (SMI) specifies rules for naming MIB variables
 - ▶ Variables named using ASN.1 (absolute, global, hierarchical)
- ▶ Simple Network Management Protocol (SNMP) specifies format of messages that pass between a manager's workstation and the managed entity
 - ▶ Message format defined with ASN.1 (similar to BNF grammar)
 - ▶ Fetch and store paradigm

